



Vue.js 컴포넌트

정의, 구조, 재사용성

목차

Vue.js 컴포넌트 여행의 시작

- 1 컴포넌트란 무엇인가?
- 2 왜 컴포넌트를 사용하는가?
- 3 SFC 구조
- 4 컴포넌트 정의 및 사용방법
- 5 재사용성 개념
- 6 실습 예제
- 7 요약

컴포넌트란 무엇인가?

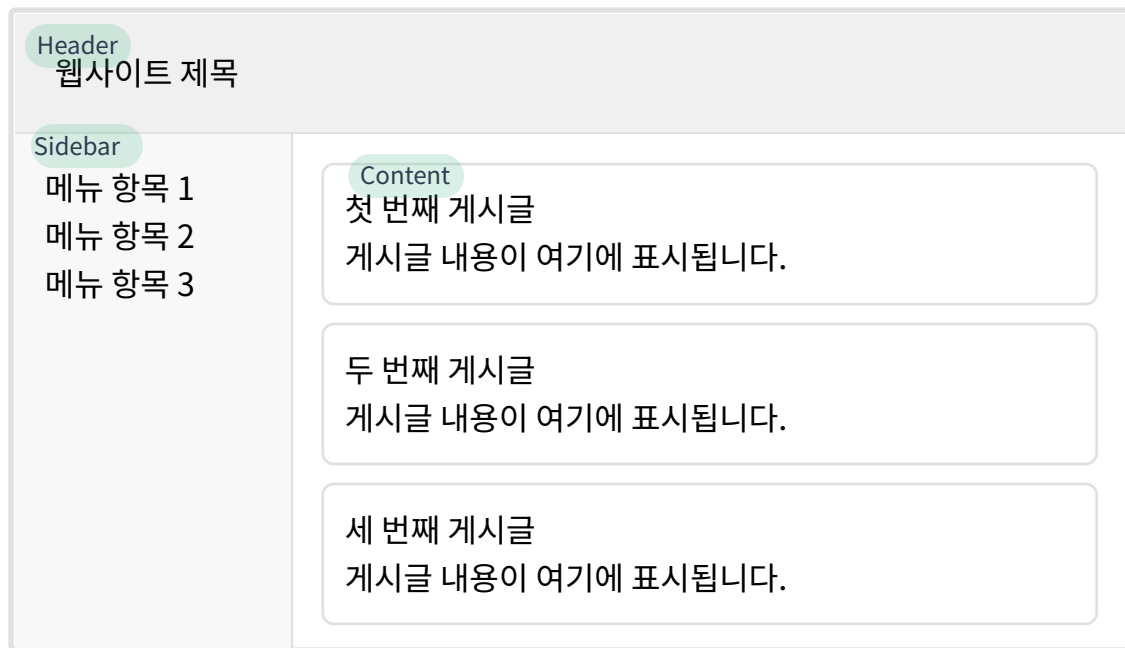
🧩 독립적이고 재사용 가능한 UI 요소

🔗 각 부분을 독립적으로 개발하고 사용 가능

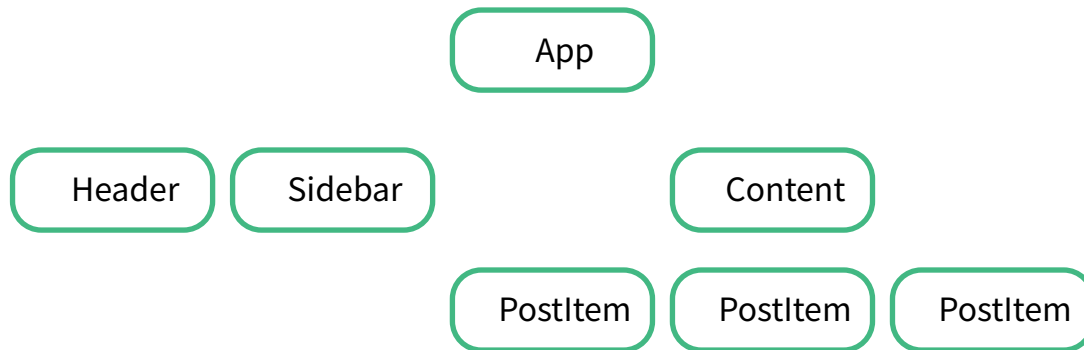
📦 템플릿(HTML), 로직(JS), 스타일(CSS)을 캡슐화

👥 중첩된 컴포넌트 트리 형태로 앱 구성

실제 웹 페이지 구조



컴포넌트 트리 구조



왜 컴포넌트를 사용하는가?



재사용성: 한 번 만들고 여러 곳에서 사용
코드 중복 감소, 개발 시간 단축



캡슐화: 독립적인 상태와 기능
컴포넌트 간 영향 최소화, 안정성 증가



확장성: 모듈식 성장
애플리케이션 규모가 커져도 구조 유지

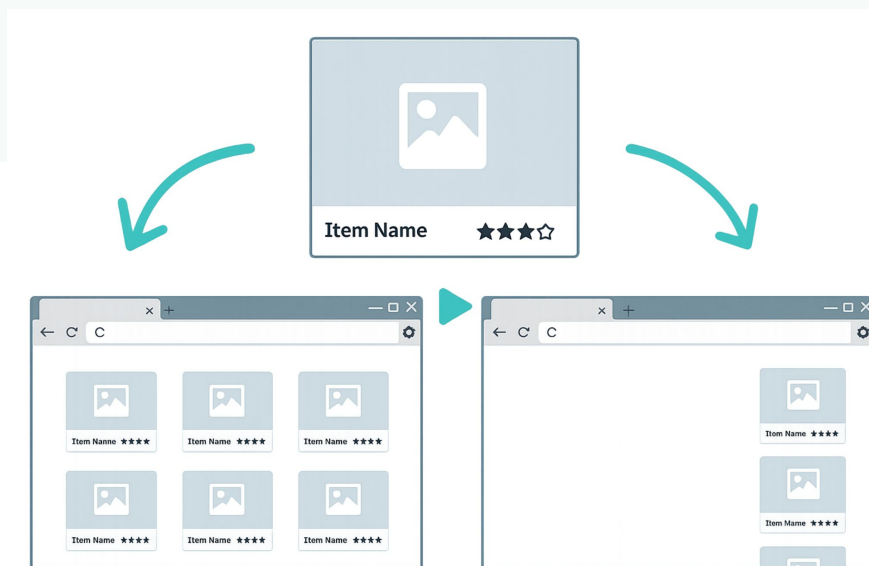


유지보수성: 작은 단위로 관리
특정 기능 업데이트 시 해당 컴포넌트만 수정



조직화: 관심사의 분리
논리적 구조화, 코드 가독성 향상

참고: 각 컴포넌트는 독립적인 인스턴스를 생성하며, 고유한 상태를 관리합니다.



SFC (Single-File Component) 구조

- 하나의 .vue 파일에 세 가지 섹션을 포함
- <template> HTML 마크업 정의
- <script> 컴포넌트 로직 및 데이터
- <style> CSS 스타일 (scoped 가능)

SFC의 장점: 모듈화, 관심사 분리, 스타일 스코핑, 전처리기 지원 (TypeScript, SCSS)

MyComponent.vue

```
<template>
  <div class="greeting">
    Hello, {{ name }}!
  </div>
</template>
```

```
<script setup>
  import { ref } from 'vue'
  // 반응형 상태 정의
  const name = ref('Vue 3')
</script>
```

```
<style scoped>
  .greeting {
    color: #42b883;
    font-weight: bold;
    margin-top: 1rem;
  }
</style>
```

컴포넌트 정의 및 사용 방법 (1/2)

자식 컴포넌트 (LikeButton.vue):

```
<script setup>
import { ref } from 'vue'

const likes = ref(0)

function handleLike() {
  likes.value += 1 // 클릭할 때마다 +1
}
</script>

<template>
  <button @click="handleLike">
    👍 {{ likes }} Like
  </button>
</template>
```

- 컴포넌트 정의: .vue 확장자를 가진 SFC(Single File Component) 파일로 작성
- 사용 방법: import로 가져와서 컴포넌트 태그로 사용

명명 규칙:

- SFC에서는 PascalCase 태그명 사용 (예: LikeButton)



컴포넌트 실습: 좋아요 버튼

각 버튼은 독립적인 상태를 가집니다.



4 Like



0 Like



0 Like

컴포넌트 정의 및 사용 방법 (2/2)

부모 컴포넌트에서의 사용:

```
<!-- src/App.vue -->
<script setup>
import LikeButton from './components/LikeButton.vue'
</script>

<template>
  <main style="text-align:center;">
    <h1>💡 컴포넌트 실습: 좋아요 버튼</h1>

    <p>각 버튼은 독립적인 상태를 가집니다.</p>

    <LikeButton />
    <LikeButton />
    <LikeButton />
  </main>
</template>
```

요약

- 컴포넌트 중심으로 사고: 작은 단위, 집중된 기능, 조합 가능하게
- 컴포넌트는 한 가지 책임
- 화면-레이아웃/데이터-표현/비즈니스-로직을 분리
SFC와 `script setup` 사용: 명확하고 간결한 코드 작성

추가 학습 리소스

Vue.js 공식 문서

<https://ko.vuejs.org/guide/essentials/component-basics>

컴포넌트는 Vue.js의 핵심입니다. 작게 시작하여 큰 애플리케이션을 구축하세요!